Dataset Generator

State Code Guide

1. Overview

The Dataset Generator ships with two interchangeable implementations of its state calculation logic. These are stored in the state_code folder and must be manually copied into the source directory before compiling the JAR. The application cannot switch between implementations at runtime — the choice is made at build time by deciding which files are placed in src/generator/.

The two implementations are:

- Blank — the default version. State variables are not calculated; the calcState() and calcStateDyn() methods do nothing. Use this when running the generator via Nostate config file only.
- Paper — the full version derived from the research paper. Required for plain, noise, and noise-and-sine run modes.

2. Folder Structure

After extracting the project, the relevant directories are:

```
generator/
  main/
    src/
      generator/
        CalcStateDyn.java      ← active file
        StateCalculator.java   ← active file
        ... (other source files)
    state_code/
      CalcStateDyn/
        CalcStateDyn_Blank.java ← blank implementation
        CalcStateDyn_Paper.java ← paper implementation
      StateCalculator/
        StateCalculator_Blank.java ← blank implementation
        StateCalculator_Paper.java java ← paper implementation
    config/
      NoState/     ← config for blank mode
      plain/       ← config for paper mode
      noise and sine/    ← config for paper mode
      only noise/ ← config for paper mode
```

Important: Never modify the _Blank or _Paper files directly. Always work on the active files in src/generator/. The state_code folder is a library of versions to copy from.

3. Which Version Do I Need?

Run Mode / Config	Required State Code Version
NoState config	Blank (default)
plain config	Paper
only noise config	Paper
noise and sine config	Paper

Note: If you only need to load a config and generate data without state-dependent variables (QCS_Moisture, QCS_BasisWeight, QCS_Caliper, etc.), you can use the Blank version and skip the replacement step entirely.

4. Default State: Blank Version

The blank version is already in place in src/generator/ when you first set up the project. You do not need to do anything to use it. Simply build the JAR as described in the JAR creation guide and run the application with the NoState config.

What the Blank version does

Both calcState() in StateCalculator.java and calcStateDyn() in CalcStateDyn.java have empty method bodies. When the generator runs, state-dependent columns will remain empty or at their default values. All other columns driven purely by config inputs will still be generated normally.

When to use Blank

- You are loading and running a config without needing derived state variables.
- You are using the NoState config folder.
- You are developing or testing and do not need the paper model outputs.

Required Input Variables

Each CalcStateDyn.java implementation contains a REQUIRED_INPUTS list that identifies the input variables needed for application-specific state calculations. The generic Generator.java uses this list to preserve the required input columns during dataset generation.

The Blank implementation uses an empty REQUIRED_INPUTS list because it performs no application-specific state calculations. This allows configurations without state variables, such as NoState, to run without requiring Paper-specific inputs.

5. Switching to the Paper Version

To enable the full paper-based state calculations required for plain, only noise, and noise and sine run modes, you must replace the two active source files with their Paper counterparts before building the JAR.

Step-by-step instructions

Step 1 — Replace CalcStateDyn.java

Copy the paper version over the active file:

```
state_code\CalcStateDyn\CalcStateDyn_Paper.java
          ↓ copy and rename
src\generator\CalcStateDyn.java
```

In practical terms: open CalcStateDyn_Paper.java, select all, copy. Open src/generator/CalcStateDyn.java, select all, paste, save.

Step 2 — Replace StateCalculator.java

Do the same for the StateCalculator:

```
state_code\StateCalculator\StateCalculator_Paper.java
          ↓ copy and rename
src\generator\StateCalculator.java
```

Important: The package declaration at the top of each Paper file already reads 'package generator;' — do not change it. The class name must remain CalcStateDyn and StateCalculator respectively for the project to compile.

Paper-Specific Input Requirements

The Paper implementation of *CalcStateDyn.java* defines the following variables in its *REQUIRED_INPUTS* list:

- MV_ThinStockFlow
- MV_ThinStockConsistency
- MV_PressLoad
- MV_SteamPressure

These variables are required for the Paper application's dynamic state calculations. Instead of identifying these variables directly in the generic generator, `Generator.java` retrieves them from the active `CalcStateDyn.REQUIRED_INPUTS` list.

When creating a new application-specific implementation, developers should update `REQUIRED_INPUTS` to include the additional input variables that must be preserved for its dynamic state calculations. Variables used only in Lab Configurations are already handled

separately by the generator.

Step 3 — Build the JAR

Follow the standard JAR creation steps from the JAR Building Guide. The Paper implementation will be compiled into the new JAR.

Step 4 — Use the matching config

Run the JAR using one of the configs that requires state calculations:

- plain
- only noise
- noise and sine

Note: You must use a matching config. Running the Paper JAR with the NoState config, or the Blank JAR with the plain config, will produce incorrect or incomplete output.

6. Reverting to the Blank Version

To go back to the blank version, repeat the same copy process using the _Blank files:

```
state_code\CalcStateDyn\CalcStateDyn_Blank.java  →
src\generator\CalcStateDyn.java
state_code\StateCalculator\StateCalculator_Blank.java →
src\generator\StateCalculator.java
```

Rebuild the JAR after replacing the files.

Important: Do not remove the PROCESS_VARIABLES Hash map from either Blank file. Both StateCalculator_Blank.java and CalcStateDyn_Blank.java declare a static Map called PROCESS_VARIABLES that must remain in place for the project to compile. CalcStateDyn copies from StateCalculator's map at startup, so removing either declaration will cause a build error. If you want to define your own process constants, add them as entries to this map in StateCalculator_Blank.java and they will automatically be available in CalcStateDyn as well.

7. Quick Reference

I want to…	Action
Run with NoState config only	Use Blank (default). No file changes needed. Build JAR and run.
Run plain / only noise / noise and sine	Copy Paper files into src/generator/. Rebuild JAR. Use matching config.
Go back to Blank after using Paper	Copy Blank files into src/generator/. Rebuild JAR.
Add my own state logic	Copy the Blank state-code templates into `src/generator/`, then add application-specific calculations to the active `CalcStateDyn.java` and `StateCalculator.java` files. Update `CalcStateDyn.REQUIRED_INPUTS` with any additional input variables required by dynamic state calculations. Ensure the selected `state.csv` contains the appropriate state variables. Rebuild the JAR after making changes.